

Complete AI Prompt — Loan Management System (Deployment Ready)

Build a production-ready full-stack Loan Management System (LMS) based on the following requirements.

Tech Stack:

- Frontend: Next.js 15 App Router + TypeScript + Tailwind CSS + ShadCN UI
- Backend: Node.js + Express.js + TypeScript
- Database: MongoDB + Mongoose
- Authentication: JWT + bcrypt
- State Management: Zustand or Redux Toolkit
- Forms: React Hook Form + Zod validation
- File Uploads: Multer + Cloudinary or local storage support
- Deployment:
 - Frontend → Vercel
 - Backend → Render/Railway
 - Database → MongoDB Atlas
- API Style: REST API
- UI Theme: Modern fintech dashboard UI with responsive mobile-first design
- Use clean architecture and scalable folder structure
- Generate production-ready code with proper environment variables and deployment setup.

PROJECT OVERVIEW

Create a Loan Management System where: 1. Borrowers can register, login, complete a multi-step loan application, upload salary slips, configure loans, and apply. 2. Internal executives can manage loans through different lifecycle stages using role-based dashboards. 3. Admin can access everything.

The complete loan lifecycle: APPLIED → SANCTIONED → DISBURSED → CLOSED

Rejected loans should also have status: REJECTED

LANDING PAGE REQUIREMENTS

Create a beautiful modern fintech landing page.

Landing page sections:

Hero Section

- Big heading: "Smart Loan Management Platform"
- Subheading: "Fast approvals, secure processing, and complete loan lifecycle management."
- CTA buttons:
 - Apply for Loan
 - Executive Login
- Modern illustration related to banking/finance
- Animated gradient background

Features Section

Show cards for:

- Easy Loan Application
- Fast Verification
- Secure Document Upload
- Live Loan Tracking
- Executive Workflow Automation
- Real-time Payment Collection

Workflow Section

Show lifecycle timeline: 1. Register 2. Submit Details 3. Upload Documents 4. Apply Loan 5. Verification 6. Sanction 7. Disbursement 8. Repayment 9. Loan Closure

Statistics Section

Show animated stats:

- 10K+ Loans Processed
- 98% Customer Satisfaction
- 24hr Approval System
- 100% Secure

Testimonials Section

Modern testimonial cards.

FAQ Section

Accordion UI.

Footer

Include:

- Quick Links
- Contact
- Social Media Icons
- Privacy Policy
- Terms

Landing page must:

- Be fully responsive
 - Use modern fintech color palette
 - Include smooth animations using Framer Motion
 - Use reusable components
 - Be SEO optimized
 - Have dark/light mode toggle
-

AUTHENTICATION SYSTEM

Implement:

- Register
- Login
- Logout
- Forgot Password
- Reset Password
- JWT Authentication
- Refresh Token Flow
- Protected Routes
- Role-based route guards

Roles:

- ADMIN
- SALES
- SANCTION
- DISBURSEMENT
- COLLECTION
- BORROWER

Passwords:

- Hash using bcrypt

- Use secure JWT cookies or Authorization headers
-

BORROWER APPLICATION FLOW

Create a step-by-step multi-step form.

Step 1 — Authentication

Borrower signs up or logs in.

Fields:

- Full Name
- Email
- Phone Number
- Password

Validation:

- Strong password
 - Valid email
 - Phone validation
-

Step 2 — Personal Details + BRE

Fields:

- Full Name
- PAN Number
- Date of Birth
- Monthly Salary
- Employment Mode
 - Salaried
 - Self-Employed
 - Unemployed
- Address
 - City
 - State
 - Pincode

Implement Business Rule Engine (BRE).

Reject if:

- Age < 23 or > 50
- Salary < 25000
- Invalid PAN format
- Employment = Unemployed

PAN Regex: `^[A-Z]{5}[0-9]{4}[A-Z]{1}$`

BRE must:

- Run on backend mandatorily
- Also run frontend validation for better UX
- Return clear error messages

Show rejected reason clearly.

Step 3 — Upload Salary Slip

Allow upload:

- PDF
- JPG
- PNG

Max Size:

- 5MB

Features:

- File preview
- Drag & Drop Upload
- Upload progress bar
- Cloudinary integration

Store:

- File URL
 - Public ID
 - Upload timestamp
-

Step 4 — Loan Configuration & Apply

Loan Amount Slider:

- ₹50,000 to ₹5,00,000

Tenure Slider:

- 30 to 365 days

Interest Rate:

- Fixed 12% per annum

Use Simple Interest formula:

$$SI = (P \times R \times T) / (365 \times 100)$$

Total Repayment = Principal + Interest

Show live calculation panel.

Display:

- Principal Amount
- Interest Amount
- Total Repayment
- Daily EMI approximation
- Due Date

When clicking Apply:

- Create loan
- Initial status = APPLIED

OPERATIONS DASHBOARD

Create separate dashboards based on role.

Each role sees only their allowed module.

Admin sees everything.

Use sidebar navigation.

Include:

- Analytics cards
 - Tables
 - Search
 - Pagination
 - Filters
 - Export CSV
 - Real-time status badges
-

DASHBOARD MODULES

SALES MODULE

Purpose:

- View registered borrowers
- Track incomplete applications
- Follow-up leads

Features:

- Borrower list
 - Application progress tracking
 - Contact details
 - Status filters
-

SANCTION MODULE

Purpose:

- Review applied loans

Features:

- View borrower details
- View salary slips
- Approve or Reject loans
- Add rejection reason
- Add sanction notes

Transitions: APPLIED → SANCTIONED APPLIED → REJECTED

DISBURSEMENT MODULE

Purpose:

- Handle sanctioned loans

Features:

- View sanctioned loans
- Mark as disbursed
- Add transaction reference
- Add disbursement date
- Upload disbursement proof

Transitions: SANCTIONED → DISBURSED

COLLECTION MODULE

Purpose:

- Record repayments

Features:

- Add payment
- View payment history
- Outstanding balance tracking
- Auto-close loans

Payment fields:

- UTR Number
- Amount
- Date

Rules:

- UTR must be unique
- Payment cannot exceed pending amount
- Auto-close loan when fully paid

Transition: DISBURSED → CLOSED

ROLE-BASED ACCESS CONTROL (RBAC)

Implement frontend + backend RBAC.

Backend:

- Auth middleware
- Role middleware
- Permission validation

Frontend:

- Route protection
- Sidebar filtering
- Unauthorized page

API must reject unauthorized access.

Use HTTP codes:

- 401 Unauthorized
 - 403 Forbidden
-

DATABASE DESIGN

Create proper Mongoose schemas.

Collections:

Users

Fields:

- name
- email
- phone
- password
- role
- isActive
- timestamps

BorrowerProfiles

Fields:

- userId
- PAN
- DOB
- salary
- employmentMode
- address
- city
- state
- pincode

Loans

Fields:

- borrowerId
- amount
- tenure
- interestRate
- interestAmount
- totalRepayment
- paidAmount
- outstandingAmount
- status
- rejectionReason
- sanctionedBy
- disbursedBy
- dates

Documents

Fields:

- borrowerId
- fileUrl
- fileType
- publicId

Payments

Fields:

- loanId

- amount
 - utrNumber
 - paymentDate
 - collectedBy
-

API REQUIREMENTS

Create complete REST APIs.

Auth APIs

POST /api/auth/register POST /api/auth/login POST /api/auth/refresh POST /api/auth/logout POST /api/auth/forgot-password POST /api/auth/reset-password

Borrower APIs

POST /api/borrower/profile POST /api/borrower/upload POST /api/borrower/apply-loan GET /api/borrower/my-loans

Loan APIs

GET /api/loans GET /api/loans/:id PATCH /api/loans/:id/sanction PATCH /api/loans/:id/reject PATCH /api/loans/:id/disburse PATCH /api/loans/:id/close

Payment APIs

POST /api/payments GET /api/payments/:loanId

Dashboard APIs

GET /api/dashboard/stats GET /api/dashboard/recent-loans

Use:

- Proper validation
 - Pagination
 - Error handling
 - Global response format
-

UI REQUIREMENTS

Use:

- ShadCN UI
- Tailwind CSS
- Framer Motion
- Lucide Icons

Pages must include:

- Skeleton loaders
- Empty states
- Error states
- Toast notifications
- Responsive tables
- Modal dialogs
- Confirmation popups

Use modern fintech aesthetics.

ADMIN PANEL

Admin can:

- View all loans
- View all users
- Manage executives
- Change roles
- View analytics
- Monitor system activity

Analytics:

- Total Loans
- Active Loans
- Closed Loans
- Total Disbursed
- Total Collections
- Monthly trends

Use charts:

- Bar chart
- Pie chart

- Line chart
-

NOTIFICATIONS SYSTEM

Implement:

- Email notifications
- In-app notifications

Send notifications for:

- Loan applied
 - Loan approved
 - Loan rejected
 - Loan disbursed
 - Payment received
 - Loan closed
-

SECURITY REQUIREMENTS

Implement:

- Helmet
 - Rate limiting
 - CORS protection
 - Input sanitization
 - Mongo injection prevention
 - XSS protection
 - Secure JWT handling
 - Environment variable management
-

PROJECT STRUCTURE

Frontend structure:

/apps/web /components /features /hooks /lib /services /store /types /app

Backend structure:

/src /config /controllers /routes /models /middlewares /services /utils /validators /scripts

SEED SCRIPT

Create a seed script that creates:

Admin: admin@lms.com / Admin@123

Sales: sales@lms.com / Sales@123

Sanction: sanction@lms.com / Sanction@123

Disbursement: disbursement@lms.com / Disbursement@123

Collection: collection@lms.com / Collection@123

Borrower: borrower@lms.com / Borrower@123

README REQUIREMENTS

Generate complete README with:

- Project overview
 - Features
 - Tech stack
 - Setup steps
 - Environment variables
 - API documentation
 - Deployment guide
 - Demo credentials
 - Screenshots section
-

DEPLOYMENT SETUP

Prepare for production deployment.

Frontend:

- Vercel configuration
- Environment variables
- Build optimization

Backend:

- Railway/Render deployment config
- CORS setup
- MongoDB Atlas connection

Include:

- Dockerfile
 - docker-compose.yml
 - .env.example
 - CI/CD GitHub Actions workflow
-

ADDITIONAL REQUIREMENTS

Implement:

- Type-safe APIs
 - Reusable components
 - Clean code
 - SOLID principles
 - Error boundaries
 - Logging system
 - API interceptors
 - Axios setup
 - Form validation
 - Protected middleware
 - Mobile responsiveness
 - Accessibility support
 - SEO optimization
-

UI DESIGN STYLE

Design inspiration:

- Stripe
- Razorpay
- Modern fintech SaaS

Use:

- Glassmorphism cards
- Gradient buttons

- Soft shadows
- Rounded corners
- Clean spacing
- Smooth transitions
- Elegant typography

Primary colors:

- Indigo
 - Blue
 - Emerald
 - White
 - Dark slate
-

FINAL OUTPUT EXPECTATION

Generate: 1. Full frontend code 2. Full backend code 3. MongoDB models 4. API routes 5. Authentication system 6. RBAC middleware 7. Multi-step loan application 8. Dashboard modules 9. Payment system 10. Deployment-ready configuration 11. Docker setup 12. Seed scripts 13. README 14. Environment variables 15. Production-ready folder structure

The project must:

- Run without errors
- Be fully responsive
- Be production ready
- Follow best coding practices
- Use TypeScript everywhere
- Include comments where needed
- Be ready for GitHub deployment
- Be ready for Vercel + Render deployment

Generate complete code now.